

Pertu

Documentación integral del sistema

Plataforma web multi-sector para ventas, gestiones e inmobiliaria

**Flask · Jinja · CSS · JavaS-
cript**

Documento técnico y funcional
Actualizado: 2026-09-30

Contenido

1	Resumen ejecutivo	3
2	Arquitectura general	3
2.1	Capas principales	3
2.2	Flujo de una petición	3
2.3	Estructura del repositorio	3
3	Acceso, usuarios y sesiones	4
3.1	Autenticación	4
3.2	Control de acceso	4
3.3	Configuración relevante	4
4	Módulo Delivery	4
4.1	Experiencia	4
4.2	Rutas	4
4.3	Reglas de negocio	5
5	Módulo Gestoría	5
6	Módulo DUAL	5
6.1	Alcance funcional	5
6.2	Plantillas principales	5
6.3	API de compra	6
7	Módulo InmoControl	6
7.1	Alcance funcional	6
7.2	Rutas de navegación	6
7.3	APIs inmobiliarias	6
8	Sistema visual y UX	7
8.1	Layouts compartidos	7
8.2	Design system transversal	7
8.3	Responsive y accesibilidad	7
9	Persistencia y modelo de datos	7
9.1	Archivos	7
9.2	Escrituras seguras actuales	7
9.3	Limitaciones del almacenamiento actual	7
10	Seguridad	7
10.1	Controles implementados	7
10.2	Pendientes antes de producción	8
11	Escalabilidad	8
11.1	Evolución recomendada	8
11.2	Diseño objetivo	8
12	Operación y mantenimiento	8
12.1	Ejecución local	8
12.2	Verificaciones recomendadas	9
13	Referencia rápida de endpoints	9
14	Conclusión	9

1 Resumen ejecutivo

Pertu es una aplicación web monolítica desarrollada con Flask que integra distintos servicios bajo una única autenticación y una experiencia visual común. El sistema organiza sus funcionalidades en cuatro sectores principales:

- **Delivery:** catálogo de milanesas, bebidas, cantidades y carrito.
- **Gestoría:** recepción y seguimiento de trámites.
- **DUAL:** tienda de indumentaria femenina y deportiva.
- **InmoControl:** catálogo inmobiliario, favoritos, consultas, tasaciones, visitas y compras.

La aplicación utiliza plantillas Jinja para renderizar HTML en el servidor, CSS modular por dominio, JavaScript vanilla para interacciones y archivos JSON como persistencia del prototipo.

Objetivo de producto

Pertu centraliza operaciones de sectores diferentes sin duplicar la autenticación, el contexto de usuario ni los mecanismos básicos de administración. La arquitectura actual es adecuada para una entrega académica o MVP y tiene puntos claros de evolución hacia servicios especializados.

2 Arquitectura general

2.1 Capas principales

Capa	Responsabilidad	Ubicación
Aplicación	Configuración Flask, rutas, validaciones y reglas de negocio	app.py
Presentación	Plantillas Jinja, layouts, formularios y navegación	templates/
Estilos	Tokens visuales, responsive, componentes por sector	static/css/
Comportamiento	Carrito, galerías, checkout, formularios y feedback	static/js/
Persistencia	Usuarios y solicitudes serializados en JSON	data/
Recursos	Imágenes de productos, propiedades, fuentes e iconos	static/img/, static/webfonts/

2.2 Flujo de una petición

1. El navegador solicita una ruta Flask.
2. Flask identifica la vista y carga el usuario desde la sesión.
3. La vista verifica rol o sector cuando corresponde.
4. Se consultan productos, propiedades o solicitudes.
5. Jinja renderiza una plantilla con el contexto necesario.
6. El navegador aplica CSS y ejecuta JavaScript para interacciones progresivas.

2.3 Estructura del repositorio

```
Pertu/
├─ app.py                # Aplicación Flask, datos de dominio y rutas
├─ data/
│   ├── users.json      # Usuarios del sistema
│   └─ requests.json    # Pedidos y solicitudes por sector
├─ templates/           # Vistas Jinja y layouts compartidos
│   ├── dual_base.html
│   ├── inmo/inmo_base.html
│   └─ legacy/          # Plantillas históricas no principales
├─ static/
│   └─ css/             # Estilos globales y por módulo
```

```

|   | js/           # Comportamiento del cliente
|   | img/          # Recursos visuales
|   | webfonts/     # Font Awesome local
|   | docs/
|   | Pertu.typ      # Esta documentación

```

3 Acceso, usuarios y sesiones

3.1 Autenticación

La ruta `/login` valida las credenciales contra `data/users.json`. Las contraseñas se comparan usando las funciones de hashing de Werkzeug. Cuando el acceso es correcto se guarda un usuario reducido en la sesión y se redirige según el rol o sector.

Los destinos principales son:

Ruta	Perfil	Destino
<code>/admin</code>	Administrador	Panel de administración
<code>/delivery</code>	Delivery	Catálogo y carrito
<code>/gestoria</code>	Gestoría	Formulario de trámite
<code>/dual</code>	DUAL	Tienda de indumentaria
<code>/inmo</code>	InmoControl	Portal inmobiliario

3.2 Control de acceso

Las funciones `dual_require_sector`, `inmo_require_sector` y `role_required` centralizan la autorización. Un usuario sin sesión vuelve a `/login`; un usuario autenticado en otro sector es redirigido a su sector correspondiente.

3.3 Configuración relevante

```

SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"
SESSION_COOKIE_SECURE = configurable mediante FLASK_COOKIE_SECURE
MAX_CONTENT_LENGTH = 1 MB
FLASK_SECRET_KEY = variable de entorno recomendada

```

Recomendación de producción

Definir siempre `FLASK_SECRET_KEY`, activar `FLASK_COOKIE_SECURE=1` detrás de HTTPS y sustituir las credenciales de ejemplo. Las sesiones y los archivos JSON son apropiados para desarrollo, pero no para múltiples procesos concurrentes.

4 Módulo Delivery

4.1 Experiencia

La página principal presenta un catálogo de milanesas con imagen, precio, cantidad, selección de bebida y cálculo de subtotal. El carrito permite modificar cantidades, quitar productos, vaciar el pedido y confirmar el checkout.

4.2 Rutas

Ruta	Función
<code>/delivery</code>	Renderiza el catálogo
<code>/carrito</code>	Muestra el carrito y el total

Ruta	Función
/vaciar	Vacía el carrito
/api/agregar	Agrega o actualiza un producto
/api/eliminar	Elimina una línea del carrito
/api/checkout	Confirma la compra

4.3 Reglas de negocio

- La cantidad máxima global definida es `MAX_QUANTITY = 20`.
- El precio se calcula en el servidor mediante `calculate_item_price`.
- El carrito se almacena en sesión.
- Las solicitudes confirmadas se persisten en `delivery_orders`.

5 Módulo Gestoría

La pantalla de gestoría ofrece una interfaz para iniciar trámites y registrar datos del solicitante. La API `/api/tramite` valida el contenido y agrega la solicitud a `gestoria_requests`. El panel administrativo permite consultar solicitudes y cambiar su estado desde `/api/admin/request-status`.

El flujo recomendado para el usuario es:

1. Elegir el tipo de trámite.
2. Completar datos personales y del vehículo o gestión.
3. Enviar la solicitud.
4. Recibir confirmación visual y número de seguimiento.

6 Módulo DUAL

6.1 Alcance funcional

DUAL es una tienda de ropa con dos líneas editoriales:

- **Mujer:** vestidos, tops, jeans, conjuntos, blusas y camperas.
- **Deportivo:** prendas y accesorios para entrenamiento.

También cuenta con categorías de ofertas y novedades, variantes de talla y color, stock por combinación, checkout, comprobante y seguimiento de envío.

6.2 Plantillas principales

Plantilla	Uso
<code>dual_base.html</code>	Layout compartido, header, navegación y footer
<code>dual_index.html</code>	Inicio, categorías destacadas y productos nuevos
<code>dual_categoria.html</code>	Listado filtrable por categoría
<code>dual_producto.html</code>	Detalle, variantes y selección de compra
<code>dual_checkout.html</code>	Datos de envío y pago
<code>dual_comprobante.html</code>	Confirmación del pedido
<code>dual_pedidos.html</code>	Historial de pedidos
<code>dual_seguimiento.html</code>	Seguimiento del envío

6.3 API de compra

POST /api/dual/pedido verifica sesión, producto, cantidad, talla, color, stock, datos de envío y método de pago. Luego genera identificadores únicos, descuenta stock, guarda el pedido en dual_orders y devuelve la URL del comprobante.

Los métodos de pago aceptados son tarjeta, Mercado Pago, transferencia y efectivo. El pedido contiene transportista, código de seguimiento, estado, total, envío y fecha de creación.

7 Módulo InmoControl

7.1 Alcance funcional

InmoControl ofrece:

- Búsqueda de propiedades para comprar o alquilar.
- Listado, detalle, galería de imágenes y mapa.
- Favoritos vinculados a la sesión.
- Solicitud de contacto y visita.
- Tasación estimada por tipo y superficie.
- Registro de intención de compra.
- Emprendimientos, servicios, novedades y cuenta.

7.2 Rutas de navegación

Ruta	Propósito
/inmo y /inmo/home	Inicio y buscador inmobiliario
/inmo/listado/<operacion>	Resultados de comprar o alquilar
/inmo/propiedad/<id>	Detalle de una propiedad
/inmo/mapa	Exploración por zonas
/inmo/emprendimientos	Proyectos inmobiliarios
/inmo/tasacion	Solicitud de tasación
/inmo/servicios	Servicios de la empresa
/inmo/novedades	Noticias y contenidos
/inmo/contacto	Contacto general
/inmo/cuenta	Favoritos y actividad del usuario

7.3 APIs inmobiliarias

Endpoint	Operación
/api/inmo/favorito	Añade o quita favoritos
/api/inmo/contacto	Registra una consulta
/api/inmo/visita	Agenda una visita
/api/inmo/tasacion	Calcula y guarda una tasación
/api/inmo/compra	Registra una intención de compra

La tasación utiliza una tarifa base por tipo de propiedad y la superficie declarada. El resultado es una estimación orientativa y no reemplaza una valuación profesional.

8 Sistema visual y UX

8.1 Layouts compartidos

Los módulos con mayor complejidad utilizan layouts base:

- `dual_base.html` para DUAL.
- `inmo/inmo_base.html` para InmoControl.

Estos layouts concentran marca, navegación, sesión, footer y bloques de extensión. De esta forma las páginas internas comparten una estructura estable y pueden cambiar el contenido sin duplicar chrome.

8.2 Design system transversal

`static/css/design-system.css` define:

- Foco visible para teclado.
- Radios y sombras base.
- Estados de controles deshabilitados.
- Compatibilidad con usuarios que prefieren reducir movimiento.

Los estilos específicos se mantienen en `dual.css`, `inmo.css`, `style.css`, `index.css` y `carrito.css` para evitar acoplar la identidad visual de un sector a otro.

8.3 Responsive y accesibilidad

- DUAL e InmoControl incluyen menú móvil con botón real, `aria-expanded`, `aria-controls` y etiqueta accesible.
- Las grillas se adaptan a una columna en pantallas pequeñas.
- Los campos y botones mantienen áreas táctiles utilizables.
- Las galerías y acciones conservan estados visuales de foco y hover.
- El sistema respeta `prefers-reduced-motion`.

9 Persistencia y modelo de datos

9.1 Archivos

Archivo	Contenido
<code>data/users.json</code>	Usuarios, roles, sectores y hashes de contraseña
<code>data/requests.json</code>	Pedidos delivery, pedidos DUAL, contactos, visitas, tasaciones y compras InmoControl

9.2 Escrituras seguras actuales

`save_json` escribe primero en un archivo temporal, fuerza el contenido a disco con `fsync` y después realiza `os.replace`. Este patrón reduce el riesgo de dejar un JSON truncado si el proceso se interrumpe durante una escritura.

9.3 Limitaciones del almacenamiento actual

- No hay transacciones ni bloqueo distribuido.
- Dos procesos pueden sobrescribir cambios simultáneos.
- Las búsquedas y filtros son lineales en memoria.
- El stock no tiene una reserva transaccional.
- Los archivos crecen sin índices ni paginación.

10 Seguridad

10.1 Controles implementados

- Hashing de contraseñas con Werkzeug.

- Sesión protegida con HttpOnly y SameSite=Lax.
- Límite de tamaño de solicitud de 1 MB.
- Validación de IDs, cantidades, variantes, stock y métodos de pago.
- Control de acceso por rol y sector.
- Headers X-Content-Type-Options, X-Frame-Options y Referrer-Policy.
- Escritura atómica de archivos JSON.

10.2 Pendientes antes de producción

1. Agregar protección CSRF para formularios y endpoints mutables.
2. Aplicar rate limiting y auditoría de intentos de login y APIs.
3. Validar y normalizar emails, teléfonos y contenido textual con mayor profundidad.
4. Mover secretos y credenciales fuera de archivos y datos de ejemplo.
5. Usar una base de datos con transacciones y permisos por usuario.
6. Agregar una política CSP compatible con las fuentes y assets reales.
7. Servir todo el tráfico mediante HTTPS y activar cookies Secure.

11 Escalabilidad

11.1 Evolución recomendada

La migración puede hacerse de forma incremental:

Etapas	Cambio	Beneficio
1	Extraer configuración y secretos a un entorno gestionado	Despliegues repetibles y seguros
2	Migrar JSON a PostgreSQL o MariaDB	Transacciones, índices y concurrencia
3	Separar catálogos y pedidos en servicios de dominio	Escalado independiente por carga
4	Incorporar Redis para sesiones y cache	Menor latencia y menos lecturas
5	Procesar emails, comprobantes y tracking con una cola	APIs más rápidas y resilientes
6	Agregar observabilidad	Métricas, logs estructurados y trazas

11.2 Diseño objetivo

La separación recomendada no implica convertir todo en microservicios de inmediato. Primero conviene mantener un monolito modular con paquetes como auth, delivery, gestoria, dual, inmo y admin. Cuando el tráfico lo justifique, los módulos con mayor carga pueden extraerse manteniendo contratos HTTP estables.

12 Operación y mantenimiento

12.1 Ejecución local

```
# Crear y activar un entorno virtual
python -m venv .venv
.venv\Scripts\activate

# Instalar dependencias del proyecto
pip install flask werkzeug

# Configurar una clave de sesión
$env:FLASK_SECRET_KEY = "clave-local-segura"
```

```
# Iniciar el servidor
python app.py
```

12.2 Verificaciones recomendadas

- Compilar sintaxis del backend con `python -m py_compile app.py`.
- Ejecutar pruebas de rutas con Flask en modo testing.
- Verificar respuestas 2xx, 4xx y redirecciones de cada sector.
- Probar checkout con stock válido, stock insuficiente y variantes inválidas.
- Probar el menú móvil con teclado y lector de pantalla.
- Ejecutar `typst compile docs/Pertu.typ docs/Pertu.pdf` para regenerar este documento.

13 Referencia rápida de endpoints

Endpoint	Método	Descripción
/login	GET/POST	Autenticación
/logout	GET	Cierre de sesión
/api/agregar	POST	Carrito Delivery
/api/eliminar	POST	Eliminación del carrito
/api/checkout	POST	Checkout Delivery
/api/tramite	POST	Alta de trámite
/api/admin/request-status	POST	Cambio de estado administrativo
/api/dual/pedido	POST	Compra DUAL
/api/inmo/favorito	POST	Favoritos inmobiliarios
/api/inmo/contacto	POST	Consulta inmobiliaria
/api/inmo/visita	POST	Solicitud de visita
/api/inmo/tasacion	POST	Tasación
/api/inmo/compra	POST	Compra inmobiliaria

14 Conclusión

Pertu cuenta con una base funcional amplia, layouts sectoriales reutilizables y validaciones de negocio concentradas en Flask. La reciente capa de diseño común mejora la coherencia visual y la experiencia responsive sin eliminar la identidad de DUAL, Delivery, Gestoría o InmoControl.

Para evolucionar hacia un sistema extremadamente escalable, la prioridad técnica es reemplazar la persistencia JSON por una base transaccional, formalizar contratos de API, incorporar pruebas automatizadas y extraer los dominios solo cuando existan métricas que justifiquen la separación. Esta estrategia conserva velocidad de desarrollo y reduce el riesgo de una migración prematura a microservicios.